Cours 03 : Les listes et les tuples

.

I. Les listes

.

1. Définition

Une liste est une **structure de données** qui contient une **série de valeurs**.
Les valeurs contenus dans une liste s'appellent aussi les **éléments** de cette liste.
Les éléments sont **modifiables** et peuvent être de **types différents**.
Une liste est déclarée par une série d'éléments séparées par des virgules et encadré par des crochets.

Exemples :

```
# Création d'une liste vide
ma_liste_vide = []

# Création d'une liste avec des strings
animaux = ["girafe","tigre","singe","souris"]

# Création d'une liste numérique
tailles = [5, 2.5, 1.75, 0.15]

# Création d'une liste avec des types différents
mixte = ["girafe",5,"souris",0.15]
```

2. Les indices

Il est possible d'**accéder à un élément dans une liste** en utilisant sa **position**. Cette position est appelée **indice** (ou index) de la liste. Il existe deux types d'indices : les positifs et les négatifs.

2.1. Indices positifs

Les indices positifs sont les plus utilisés pour accéder à un élément dans une liste.
Les indices positifs d'une liste de ***n*** éléments commencent à ***0*** et se terminent à ***n - 1***.

Modèle :

```
animaux = ["girafe","tigre","singe","souris"]
indice  :      0       1       2       3
# La liste animaux possède 4 (n) éléments et les indices commencent à 0 et
terminent à 3 (n-1).
```

Exemples d'utilisation :

```
print(animaux[0])
# Affiche : girafe

print(animaux[3])
# Affiche : souris

print(animaux[4])
# Erreur : IndexError: list index out of range
# L'indice 4 n'existe pas dans cette liste.
```

2.2. Indices négatifs

Les indices négatifs reviennent à **compter à partir de la fin**.
Leur principal avantage est que vous pouvez accéder au **dernier élément** d'une liste à l'aide de l'**indice -1** sans pour autant connaître la longueur de cette liste.

Modèle :

```
animaux        = ["girafe","tigre","singe","souris"]
indice positif :      0       1       2       3
indice negatif :     -4      -3      -2      -1
```

Exemples d'utilisation :

```
print(animaux[-1])    # Affiche : souris
print(animaux[-2])    # Affiche : singe
```

2.3. Tranche d'indices

Il est possible de **sélectionner une partie** d'une liste en utilisant un indiçage construit sur le modèle ***[m:n+1]*** pour **récupérer** tous les éléments, du émième au énième (**de l'élément m inclus à l'élément n+1 exclu**). On dit alors qu'on récupère une **tranche** de la liste.

Exemples :

```
animaux = ["girafe","tigre","singe","souris"]
print(animaux[0:2])   # ['girafe', 'tigre']
print(animaux[1:4])   # ['tigre', 'singe', 'souris']
print(animaux[2:-1])  # ['singe']
```

Lorsqu'**aucun indice** n'est indiqué à **gauche ou à droite** du symbole **deux-points :** , Python prend par défaut **tous les éléments** depuis le **début** ou tous les éléments jusqu'à la **fin** respectivement.

Exemples :

```
print(animaux[0:])    # ['girafe', 'tigre', 'singe', 'souris']
print(animaux[1:])    # ['tigre', 'singe', 'souris']
print(animaux[:])     # ['girafe', 'tigre', 'singe', 'souris']
```

On peut aussi préciser le **pas** en ajoutant un symbole deux-points supplémentaire et en indiquant le pas par un entier.

> Syntaxe d'une tranche d'indice : ***liste[début : fin : pas]***

Exemples :

```
animaux = ["girafe","tigre","singe","souris"]
print(animaux[0:3:2])     # ['girafe', 'singe']

chiffres = [0, 1, 2, 3, 4, 5, 6 , 7, 8, 9]
print(chiffres[::2])      # [0, 2, 4, 6, 8]
print(chiffres[::3])      # [0, 3, 6, 9]
print(chiffres[1:6:3])    # [1, 4]
```

.

3. Opérations sur les listes

3.1. La concaténation et la duplication

Tout comme les chaînes de caractères, les listes supportent l'**opérateur** **+** de **concaténation**, ainsi que l'**opérateur** ***** pour la **duplication**.

Exemples :

```
ani1 = ["girafe","tigre"]
ani2 = ["singe","souris"]
print(ani1 + ani2)  # ['girafe', 'tigre', 'singe', 'souris']
print(ani1 * 3)     # ['girafe', 'tigre', 'girafe', 'tigre', 'girafe', 'tigre']
```

La concaténation est pratique mais pour ajouter un élément à la fin d'une liste il est possible d'utiliser la **méthode .append() plus souvent utilisée que la concaténation**. (Voir plus loins dans le cours sur les méthodes avec les listes).

3.2. Modifier un élément dans une liste

Les listes sont des structures de données mutables. C'est-à-dire que l'on peut modifier les éléments dans une liste. Il existe plusieurs méthodes associées aux listes qui permettent d'apporter des modifications sur les éléments de la liste. Il est aussi possible de modifier une seule valeur dans une liste en utilisant la syntaxe suivante :

> Syntaxe : ***name_list[index] = new_value***

Exemples :

```
lettres = ["A", "B", "B", "D"]
lettres[2] = "C"
print(lettres)
# ['A', 'B', 'C', 'D']
```

3.3. Test d'appartenance

L'opérateur **in** teste si un élément fait partie d'une liste.
Il est possible de tester si un élément ne fait pas partie d'une liste en utilisant **not in**.
Ces tests renvoient un **booléen** : **True** ou **False**.

> Syntaxe : ***value (not) in name_list***

Exemples :

```
liste1 = [1, 3, 5, 7, 9]
print(3 in liste1)      # True
print(4 in liste1)      # False
print(3 not in liste1)  # False
print(4 not in liste1)  # True
```

.

4. Les fonctions avec les listes

4.1. La fonction len()

La fonction **len()** permet de connaître la longueur d'une liste, c'est-à-dire le **nombre d'éléments** dans la liste.

> Syntaxe : ***len(name_list)***

Exemples :

```
animaux = ["girafe","tigre","singe","souris"]
print(len(animaux))       # 4

print(len([1,2,3,4,5,6,7,8]))    # 8
```

4.2. Les fonctions list() et range()

La fonction **range()** permet de **générer automatiquement une série de nombres entiers**.

Par défaut, la fonction range() génère un tuple. Pour **générer une liste** il est donc nécessaire de la **combiner avec** la fonction **list()**.

La fonction range() permet de générer une série de nombres entiers compris entre :

- 0 (inclu) et fin (exclu) : **[0 : fin[**

> Syntaxe : ***range(fin)***

Exemple :

```
print(list(range(10)))
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- début (inclu) et fin (exclu) : **[début : fin[**

> Syntaxe : ***range(début, fin)***

Exemples :

```
print(list(range(1,5)))
# [1, 2, 3, 4]

print(list(range(15,20)))
# [15, 16, 17, 18, 19]
```

- début (inclu) et fin (exclu) par saut de pas : **[début : fin[+ pas**

> Syntaxe : ***range(début, fin, pas)***

Exemples :

```
print(list(range(0,1000,200)))
# [0, 200, 400, 600, 800]

print(list(range(2,-2,-1)))
# [2, 1, 0, -1]
```

4.3. Les fonctions min() max() et sum()

Les fonctions **min()**, **max()** et **sum()** renvoient respectivement le **minimum**, le **maximum** et la **somme** d'une **liste numérique** passée en argument.

Exemples :

```
chiffres = list(range(10))
print(chiffres)         # [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(max(chiffres))    # 9
print(min(chiffres))    # 0
print(sum(chiffres))    # 45
```

.

5. Les méthodes avec les listes

.

> **Notion : une méthode en Python**
> En Python, on peut considérer chaque variable comme un objet sur lequel on peut appliquer des méthodes. Une méthode est simplement **une fonction qui utilise et/ou agit sur l'objet** lui-même, les deux étant connectés par un point.
> **Syntaxe : objet.methode()**

5.1. Afficher la position d'un élément : .index()

Pour **connaître l'indice** (la position) **d'un élément** dans une liste on utilise la méthode **.index()**

> Syntaxe : ***name_list.index(value)***

Exemples :

```
lettres = ["A", "B", "C"]
print(lettres.index("B"))   # 1

chiffres = [1, 2, 3, 4, 5]
print(chiffres.index(4))    # 3
```

5.2. Ajouter : .append()

La méthode **.append()** permet d'**ajouter un élément à la fin** d'une liste.

> Syntaxe : ***name_list.append(new_value)***

Exemple :

```
chiffres = [1, 2, 3]
chiffres.append(5)
print(chiffres)
# [1, 2, 3, 5]
```

5.3. Supprimer : .remove() - .pop() - del - .clear()

- La méthode **.remove()** permet de supprimer un élément d'une liste à partir de sa valeur.

> Syntaxe : ***name_list.remove(value)***

Exemple :

```
lettres = ["A", "B", "C", "D"]
lettres.remove("B")
print(lettres)
# ['A', 'C', 'D']
```

.

- La méthode **.pop()** permet de supprimer un élément d'une liste à partir de sa position (indice).

> Syntaxe : ***name_list.pop(index)***

Exemple :

```
lettres = ["A", "B", "C", "D"]
lettres.pop(0)
print(lettres)
# ['B', 'C', 'D']
```

.

- L'instruction **del** permet également de supprimer un élément à partir de son indice.

> Syntaxe : ***del name_liste[index]***

Exemple :

```
lettres = ["A", "B", "C", "D"]
del lettres[1]
print(lettres)
# ['A', 'C', 'D']
```

.

- La méthode **.clear()** permet de supprimer tous les éléments d'une liste. La liste n'est pas supprimée, elle est vide.

> Syntaxe : ***name_list.clear()***

Exemple :

```
lettres = ["A", "B", "C", "D"]
lettres.clear()
print(lettres)
# []
```

5.4. Insérer : .insert()

La méthode **.insert()** permet d'**insérer un élément à une position** (indice) donnée.

> Syntaxe : ***name_list.insert(index, value)***

Exemple :

```
lettres = ["A", "B", "C", "D"]
lettres.insert(2, "X")
print(lettres)
# ['A', 'B', 'X', 'C', 'D']
```

5.5. Inverser : .reverse()

La méthode **.reverse()** permet d'**inverser les éléments** d'une liste.

> Syntaxe : ***name_list.reverse()***

Exemple :

```
lettres = ["A", "B", "C", "D"]
lettres.reverse()
print(lettres)
# ['D', 'C', 'B', 'A']
```

5.6. Trier : .sort - sorted()

La méthode **.sort()** permet de **trier les éléments** d'une liste par **ordre croissant** (du plus petit au plus grand).

> Syntaxe : ***name_list.sort()***

Exemple :

```
chiffres = [4, 9, 3, 7, 2, 1]
chiffres.sort()
print(chiffres)
# [1, 2, 3, 4, 7, 9]
```

Il est possible de **trier par ordre décroissant** (du plus grand au plus petit) en utilisant l'argument ***reverse = True***.

> Syntaxe : ***name_list.sort(reverse = True)***

Exemple :

```
chiffres = [4, 9, 3, 7, 2, 1]
chiffres.sort(reverse = True)
print(chiffres)
# [9, 7, 4, 3, 2, 1]
```

La fonction **sorted()** permet également de trier une liste mais contrairement à la méthode .sort(), cette fonction renvoie la liste triée et ne modifie pas la liste initiale.

> Syntaxe : ***sorted(name_list)***

Exemple :

```
chiffres = [4, 9, 3, 7, 2, 1]
print(sorted(chiffres))
# [1, 2, 3, 4, 7, 9]

print(chiffres)
# [4, 9, 3, 7, 2, 1]
```

5.7. Compter : .count()

La méthode **.count()** permet de **compter le nombre d'occurence** d'un élément dans une liste. C'est-à-dire, le nombre de répétition de l'élément dans la liste.

> Syntaxe : ***name_list.count(value)***

Exemple :

```
chiffres = [2, 3, 1, 2, 1, 1]
print(chiffres.count(1))
# 3
```

5.8. Copier : .copy()

La méthode **.copy()** permet de **copier** une liste.

> Syntaxe : ***new_list = name_list.copy()***

Exemple :

```
chiffres = [2, 3, 1, 2, 1, 1]
new_chiffres = chiffres.copy()
print(new_chiffres)
# [2, 3, 1, 2, 1, 1]
```

5.9. Convertion avec un string : .split() - .join()

- **Conversion string en list** : en utilisant la méthode **.split()**

> Syntaxe : ***name_list.split(séparateur)***

Exemple :

```
mon_string = "Prénom:NOM:Ville"
print(mon_string.split(":"))
# ['Prénom', 'NOM', 'Ville']
```

.

- **Conversion d'une liste en string** : en utilisant la méthode **.join()**

> Syntaxe : ***"séparateur".join(name_list)***

Exemple :

```
lettres = ["A", "B", "C", "D"]
print("-".join(lettres))
# A-B-C-D
```

.

6. Liste de listes

Les listes peuvent être utilisées comme élément dans une autre liste, on appelle cela une **liste de listes**.

Exemple :

```
prairie1 = ["girafe", 4]
prairie2 = ["tigre", 2]
prairie3 = ["singe", 5]
savane = [prairie1, prairie2, prairie3]
print(savane)
# [['girafe', 4], ['tigre', 2], ['singe', 5]]
```

Pour accéder à **un élément de la liste**, on utilise l'indiçage habituel :

```
print(savane[1])
# ['tigre', 2]
```

Pour accéder à un **élément** de la **sous-liste**, on utilise un **double indiçage** :

```
print(savane[1][0])
# tigre

print(savane[1][1])
# 2
```

Remarque : Il existe une autre structure de données permettant de mieux stocker les données structurées : les **dictionnaires**.

.

II. Les tuples

Les tuples sont une **structure de données** spécifique au language Python.
Les tuples sont très similaires aux listes d'un point de vue de leur utilisations mais les tuples sont **immuables** : **les éléments ne peuvent pas être modifiés**.
Un tuple est déclaré par une série d'éléments séparées par des virgules et encadré par des **parenthèses**.

Exemples :

```
# Création d'un tuple vide
mon_tuple_vide = ()

# Création d'un tuple avec des strings
animaux = ("girafe","tigre","singe","souris")

# Création d'un tuple numérique
tailles = (5, 2.5, 1.75, 0.15)

# Création d'un tuple avec des types différents
mixte = ("girafe",5,"souris",0.15)
```